COURSE CODE : CSA13
COURSE NAME : THEORY OF COMPUTATION

1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a.

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 int state = 0; /* 0=start, 1=started with a and ends a,
 2=started with a and ends b, 3=dead */

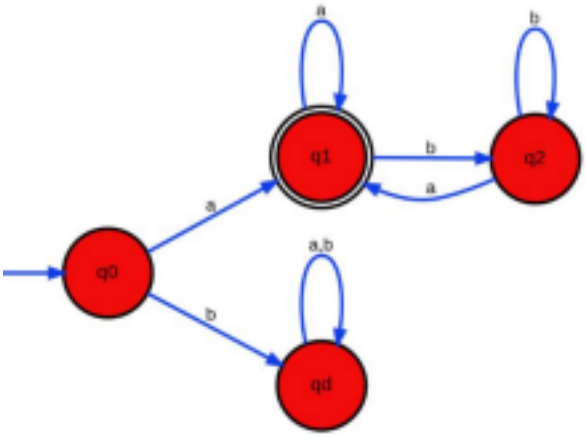
 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }

 switch (state) {
 case 0:
 state = (s[i] == 'a') ? 1 : 3;
 break;
 case 1:
 state = (s[i] == 'a') ? 1 : 2;
 break;
 case 2:
 state = (s[i] == 'a') ? 1 : 2;
 break;
 case 3:
 break;
 }
 }

 if (state == 1) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation output:



2. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with 0 and end with 1

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 int state = 0; /* 0=start, 1=starts 0/ends 0,
 2=starts 0/ends 1, 3=dead */

 printf("Enter a binary string: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
```

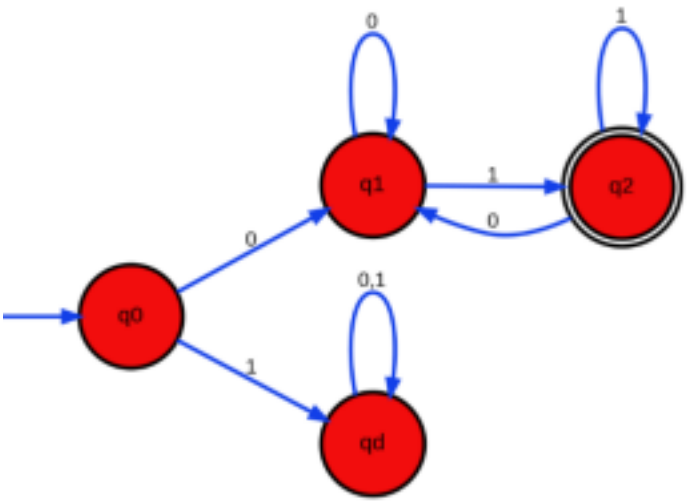
```
s[strcspn(s, "\\n")] = '\\0';

for (int i = 0; s[i] != '\\0'; i++) {
if (s[i] != '0' && s[i] != '1') {
printf("Invalid input\\n");
return 0;
}
switch (state) {
case 0: state = (s[i] == '0') ? 1 : 3; break;
case 1: state = (s[i] == '0') ? 1 : 2; break;
case 2: state = (s[i] == '0') ? 1 : 2; break;
case 3: break;
}
}

if (state == 2) printf("String Accepted\\n");
else printf("String Rejected\\n");
return 0;
}
en(str)-1] == '1')
 printf("String Accepted\n");
 else
 printf("String Rejected\n");

 return 0;
}
```

Simulation output:



3. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with b and end with a

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 int current = 1; /* bit 0: q0, bit 1: q1, bit 2: q2 */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

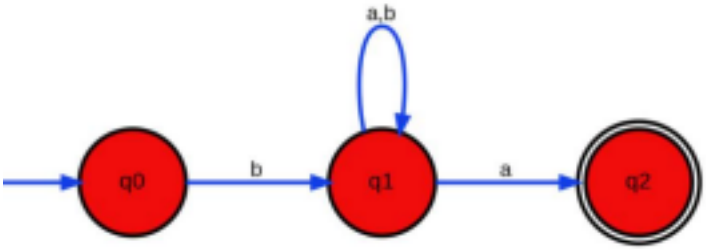
 for (int i = 0; s[i] != '\\0'; i++) {
 int next = 0;
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }

 if (current & 1) { /* q0 --b--> q1 */
 if (s[i] == 'b') next |= 2;
 }
 if (current & 2) { /* q1 loops on a,b; on a also goes q2 */
```

```
next |= 2;
if (s[i] == 'a') next |= 4;
}
current = next;
}

if (current & 4) printf("String Accepted\\n");
else printf("String Rejected\\n");
return 0;
}
```

Simulation output:



4. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with o and end with 1

```
#include <stdio.h>
#include <string.h>

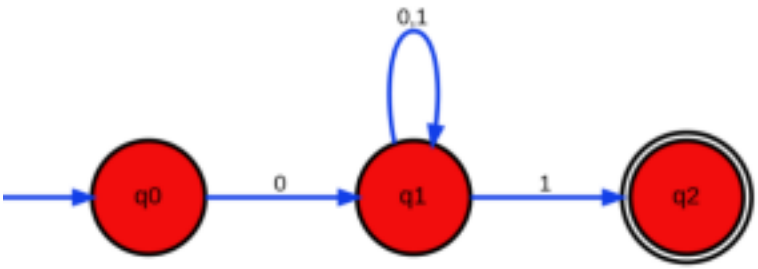
int main(void) {
 char s[100];
 int current = 1; /* bit 0: q0, bit 1: q1, bit 2: q2 */

 printf("Enter a binary string: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 int next = 0;
 if (s[i] != '0' && s[i] != '1') {
 printf("Invalid input\\n");
 return 0;
 }
 if (current & 1) {
 if (s[i] == '0') next |= 2;
 }
 if (current & 2) {
 next |= 2;
 if (s[i] == '1') next |= 4;
 }
 current = next;
 }

 if (current & 4) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation output:



5. Design a Deterministic Finite Automaton using a simulator to accept an even number of a's.

```
#include <stdio.h>
#include <string.h>
```

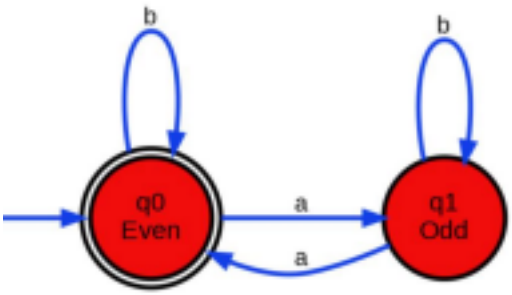
```
int main(void) {
 char s[100];
 int state = 0; /* 0=even (start/accept), 1=odd */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] == 'a') state = 1 - state;
 else if (s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }
 }

 if (state == 0) printf("String Accepted (even number of a's)\\n");
 else printf("String Rejected (odd number of a's)\\n");
 return 0;
}
```

Simulation Output:



6. Design Deterministic Finite Automata using simulator to accept odd number of a's

```
#include <stdio.h>
#include <string.h>

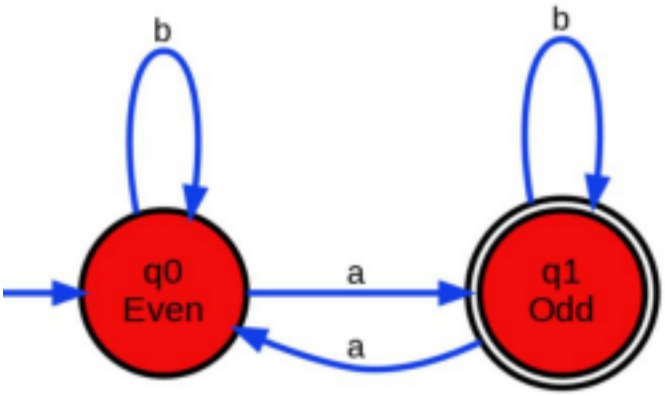
int main(void) {
 char s[100];
 int state = 0; /* 0=even, 1=odd (accept) */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] == 'a') state = 1 - state;
 else if (s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }
 }

 if (state == 1) printf("String Accepted (odd number of a's)\\n");
 else printf("String Rejected (even number of a's)\\n");
 return 0;
}
```

Simulation Output:



7. Design Deterministic Finite Automata using simulator to accept the string the end with ab over set {a,b)

```
#include <stdio.h>
#include <string.h>

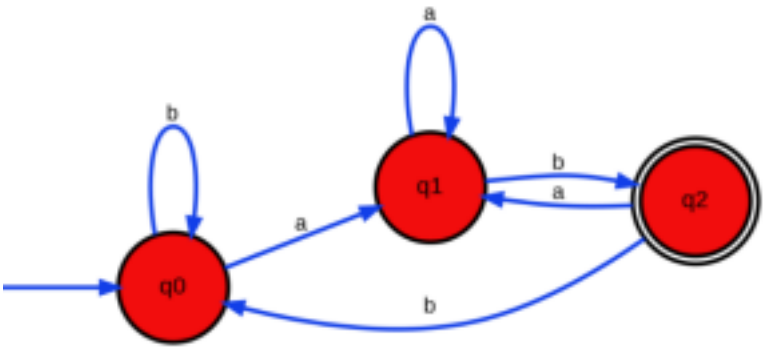
int main(void) {
 char s[100];
 int state = 0; /* 0=no useful suffix, 1=suffix a, 2=suffix ab */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] == 'a') state = 1;
 else if (s[i] == 'b') state = (state == 1) ? 2 : 0;
 else {
 printf("Invalid input\\n");
 return 0;
 }
 }

 if (state == 2) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation output:



8. Design Deterministic Finite Automata using simulator to accept the string having 'ab' as substring over the set {a,b}

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 int state = 0; /* 0=not found, 1=just saw a, 2=found ab */

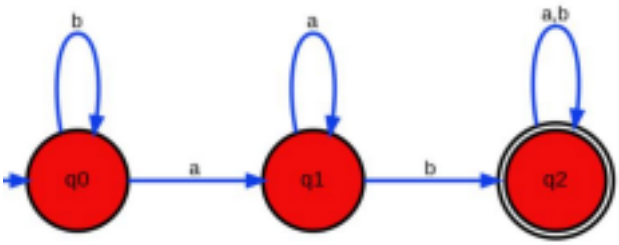
 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
```

```
}
if (state == 0) state = (s[i] == 'a') ? 1 : 0;
else if (state == 1) state = (s[i] == 'b') ? 2 : 1;
else state = 2;
}

if (state == 2) printf("String Accepted\\n");
else printf("String Rejected\\n");
return 0;
}
```

Simulation Output:



9. Draw a Deterministic Finite Automata for the language accepting strings ending with 'abba' over input alphabets Σ = {a, b}

```
#include <stdio.h>
#include <string.h>

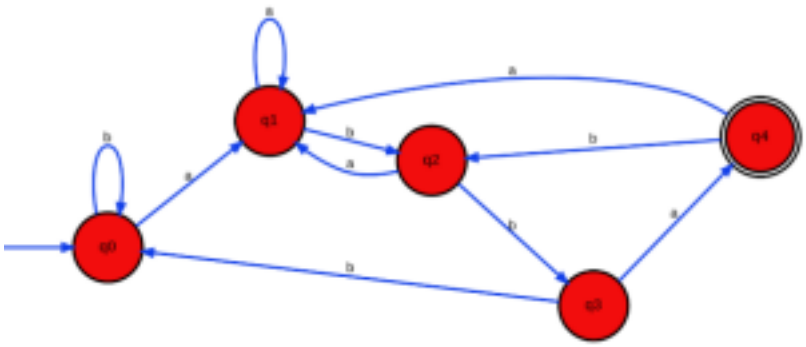
int main(void) {
 char s[100];
 int state = 0; /* 0=not found, 1=just saw a, 2=found ab */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }
 if (state == 0) state = (s[i] == 'a') ? 1 : 0;
 else if (state == 1) state = (s[i] == 'b') ? 2 : 1;
 else state = 2;
 }

 if (state == 2) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:



10. Write a C program to find ε -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ε -moves.

```
#include <stdio.h>

#define MAX 10

int main(void) {
 int n, reach[MAX][MAX] = {0};
```

```
 printf("Enter number of states (maximum %d): ", MAX);
 if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
 printf("Invalid number of states\\n");
 return 1;
 }

 printf("Enter the epsilon-transition adjacency matrix (%d x %d):\\n", n, n);
printf("Enter 1 if q(i) has an epsilon transition to q(j), otherwise 0.\\n");  for (int i
= 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 if (scanf("%d", &reach[i][j]) != 1 ||
 (reach[i][j] != 0 && reach[i][j] != 1)) {
 printf("Invalid matrix value\\n");
 return 1;
 }
 }
 reach[i][i] = 1; /* Every state belongs to its own epsilon-closure. */
 }

 /* Transitive closure: include paths containing any number of epsilon edges. */  for
(int k = 0; k < n; k++)
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);

 printf("\\nEpsilon-closures:\\n");
 for (int i = 0; i < n; i++) {
 printf("E-closure(q%d) = { ", i);
 for (int j = 0; j < n; j++)
 if (reach[i][j]) printf("q%d ", j);
 printf("}\\n");
 }
 return 0;
}
```

Simulation Output:

11. Write a C program to find ε -closure for all the states in a Non Deterministic Finite Automata (NFA) with ε -moves.

```
#include <stdio.h>

#define MAX 10

int main(void) {
 int n, reach[MAX][MAX] = {0};

 printf("Enter number of states (maximum %d): ", MAX);
 if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
 printf("Invalid number of states\\n");
 return 1;
 }

 printf("Enter the epsilon-transition adjacency matrix (%d x %d):\\n", n, n);
printf("Enter 1 if q(i) has an epsilon transition to q(j), otherwise 0.\\n");  for (int i
= 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 if (scanf("%d", &reach[i][j]) != 1 ||
 (reach[i][j] != 0 && reach[i][j] != 1)) {
 printf("Invalid matrix value\\n");
 return 1;
 }
 }
 reach[i][i] = 1; /* Every state belongs to its own epsilon-closure. */
 }

 /* Transitive closure: include paths containing any number of epsilon edges. */  for
(int k = 0; k < n; k++)
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);

 printf("\\nEpsilon-closures:\\n");
 for (int i = 0; i < n; i++) {
 printf("E-closure(q%d) = { ", i);
 for (int j = 0; j < n; j++)
 if (reach[i][j]) printf("q%d ", j);
 printf("}\\n");
 }
 return 0;
}
```

Simulation Output:

12. Design Deterministic Finite Automata using simulator to accept the input string “a” ,”ac”,and ” bac”.

```
#include <stdio.h>
#include <string.h>

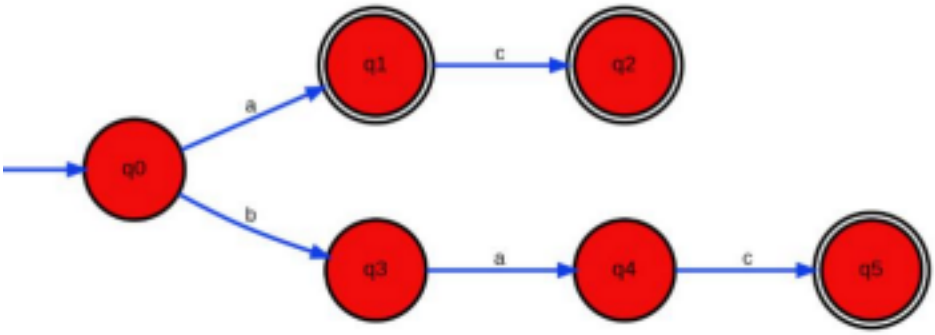
int main(void) {
 char s[100];
 int state = 0;
 /* Trie DFA:
 q0 --a--> q1(accept) --c--> q2(accept)
 q0 --b--> q3 --a--> q4 --c--> q5(accept).
 All unspecified transitions go to a dead state. */

 printf("Enter a string over {a,b,c}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 switch (state) {
 case 0: state = (s[i] == 'a') ? 1 : (s[i] == 'b') ? 3 : -1; break;
 case 1: state = (s[i] == 'c') ? 2 : -1; break;
 case 3: state = (s[i] == 'a') ? 4 : -1; break;
 case 4: state = (s[i] == 'c') ? 5 : -1; break;
 default: state = -1; break;
 }
 }

 if (state == 1 || state == 2 || state == 5)
 printf("String Accepted\\n");
 else
 printf("String Rejected\\n");
 return 0;
```

Simulation Output:



13. Design DFA using simulator to accept the string the end with ab over set {a,b)

W= abbaabab

```
#include <stdio.h>
#include <string.h>

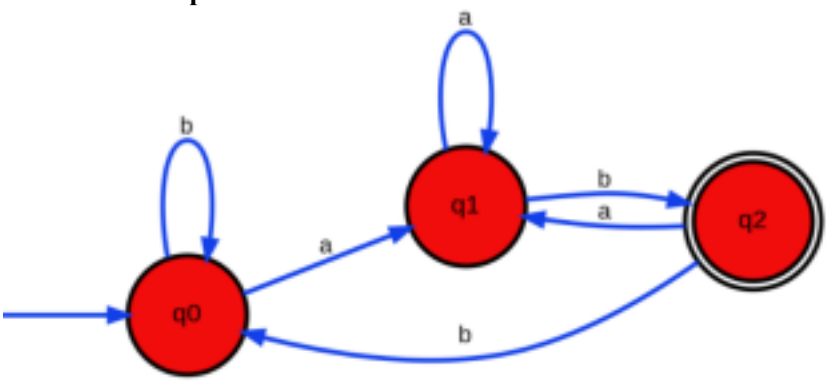
int main(void) {
 char s[100];
 int state = 0; /* 0=no useful suffix, 1=suffix a, 2=suffix ab */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] == 'a') state = 1;
 else if (s[i] == 'b') state = (state == 1) ? 2 : 0;
 else {
 printf("Invalid input\\n");
 return 0;
 }
 }

 if (state == 2) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:



14. Design Deterministic Finite Automata using simulator to accept the input string "bc" ,"c",and " bcaaa".

```
#include <stdio.h>
#include <string.h>

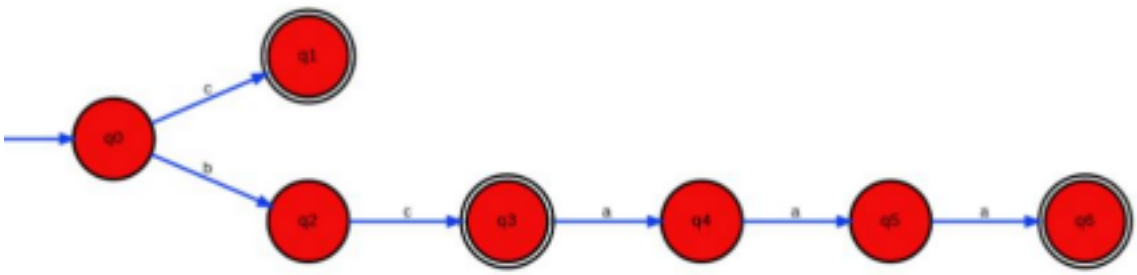
int main(void) {
 char s[100];
 int state = 0;
 /* q0 --c--> q1(accept)
 q0 --b--> q2 --c--> q3(accept) --a--> q4 --a--> q5 --a--> q6(accept) */

 printf("Enter a string over {a,b,c}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 switch (state) {
 case 0: state = (s[i] == 'b') ? 2 : (s[i] == 'c') ? 1 : -1; break;
 case 2: state = (s[i] == 'c') ? 3 : -1; break;
 case 3: state = (s[i] == 'a') ? 4 : -1; break;
 case 4: state = (s[i] == 'a') ? 5 : -1; break;
 case 5: state = (s[i] == 'a') ? 6 : -1; break;
 default: state = -1; break;
 }
 }

 if (state == 1 || state == 3 || state == 6)
 printf("String Accepted\\n");
 else
 printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:



15. Draw a Deterministic Finite Automata for the language accepting strings ending with '01' over input alphabets ∑ = {0, 1}.

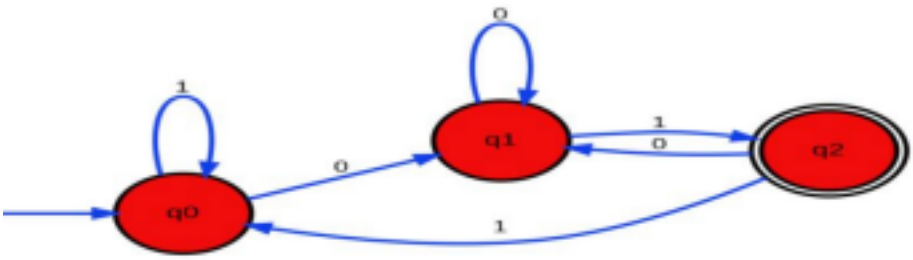
```
#include <stdio.h>
#include <string.h>
int main(void) {
 char s[100];
 int state = 0; /* 0=no trailing 0, 1=last symbol 0, 2=ends 01 */

 printf("Enter a binary string: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';
```

```
for (int i = 0; s[i] != '\\0'; i++) {
if (s[i] == '0') state = 1;
else if (s[i] == '1') state = (state == 1) ? 2 : 0;
else {
printf("Invalid input\\n");
return 0;
}
}

if (state == 2) printf("String Accepted\\n");
else printf("String Rejected\\n");
return 0;
}
}
printf("String Accepted\n");
return 0;
}
```

Simulation Output:



16. Design Deterministic Finite Automata using simulator to accept strings in which a's always appear tripled over input {a,b}

```
#include <stdio.h>
#include <string.h>

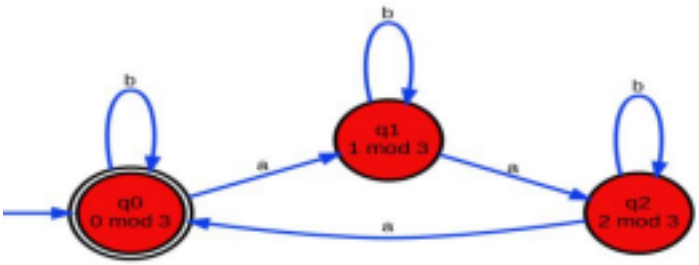
int main(void) {
 char s[100];
 int state = 0; /* remainder of count(a) divided by 3: 0, 1 or 2 */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] == 'a') state = (state + 1) % 3;
 else if (s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }
 }

 if (state == 0) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:



17. Design Non Deterministic Finite Automata using simulator to accept the string the start with a and end with b over set {a,b} and check W= abaab is accepted or not.

```
#include <stdio.h>
#include <string.h>
```

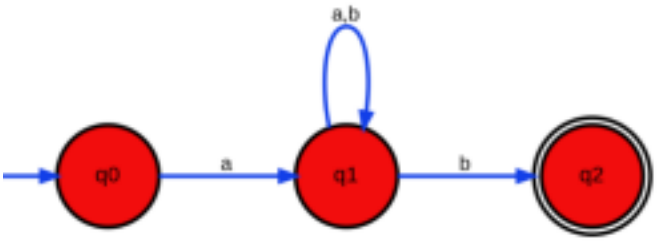
```
int main(void) {
 char s[100];
 int current = 1; /* bit 0=q0, bit 1=q1, bit 2=q2 */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 int next = 0;
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }
 if (current & 1) {
 if (s[i] == 'a') next |= 2; /* first symbol must be a */
 }
 if (current & 2) {
 next |= 2; /* q1 loops on a and b */
 if (s[i] == 'b') next |= 4; /* guess this b is the final symbol */
 }
 current = next;
 }

 if (current & 4) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:



18. Design Non Deterministic Finite Automata using simulator to accept the string that start and end with different symbols over the input {a,b}.

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 /* NFA states:
 q0=start; q1=first symbol was a; q2=accept after final b;
 q3=first symbol was b; q4=accept after final a.
 q1 and q3 loop over either symbol and nondeterministically branch
 to the accepting state when the opposite final symbol is read. */
 int current = 1; /* bit 0=q0, bit 1=q1, bit 2=q2, bit 3=q3, bit 4=q4 */

 printf("Enter a string over {a,b}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';
 for (int i = 0; s[i] != '\\0'; i++) {
 int next = 0;
 if (s[i] != 'a' && s[i] != 'b') {
 printf("Invalid input\\n");
 return 0;
 }

 if (current & 1) {
 if (s[i] == 'a') next |= 2; /* first symbol a */
 else next |= 8; /* first symbol b */
 }
 if (current & 2) { /* first symbol a */
 next |= 2; /* consume more middle symbols */
 if (s[i] == 'b') next |= 4; /* choose final b */
 }
 if (current & 8) { /* first symbol b */
 next |= 8;
 if (s[i] == 'a') next |= 16;/* choose final a */
 }
 current = next;
```

```
}

if (current & (4 | 16)) printf("String Accepted\\n");
else printf("String Rejected\\n");
return 0;
}
```

Simulation Output:

19. Draw a Deterministic Finite Automata that accepts a language L over input alphabets ∑ = {0, 1} such that L is the set of all strings starting with ’00’.

```
#include <stdio.h>
#include <string.h>

int main(void) {
 char s[100];
 int state = 0; /* 0=start, 1=first 0 seen, 2=prefix 00 confirmed, 3=dead */

 printf("Enter a binary string: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 if (s[i] != '0' && s[i] != '1') {
 printf("Invalid input\\n");
 return 0;
 }
 switch (state) {
 case 0: state = (s[i] == '0') ? 1 : 3; break;
 case 1: state = (s[i] == '0') ? 2 : 3; break;
 case 2: break; /* any remaining 0 or 1 is allowed */
 case 3: break;
 }
 }

 if (state == 2) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation Output:

20. Design DFA using simulator to accept the string the end with abc over set {a,b,c) W= abbaababc

```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
 char s[100];
 int state = 0;
 /* States are the longest suffix matching a prefix of "abc":
 0="", 1="a", 2="ab", 3="abc" (accept). */
 int trans[4][3] = {
 {1, 0, 0}, /* state 0 on a,b,c */
 {1, 2, 0}, /* state 1 */
 {1, 0, 3}, /* state 2 */
 {1, 0, 0} /* state 3 */
 };

 printf("Enter a string over {a,b,c}: ");
 if (!fgets(s, sizeof(s), stdin)) return 1;
 s[strcspn(s, "\\n")] = '\\0';

 for (int i = 0; s[i] != '\\0'; i++) {
 int col;
 if (s[i] == 'a') col = 0;
 else if (s[i] == 'b') col = 1;
 else if (s[i] == 'c') col = 2;
 else {
 printf("Invalid input\\n");
 return 0;
 }
 state = trans[state][col];
 }

 if (state == 3) printf("String Accepted\\n");
 else printf("String Rejected\\n");
 return 0;
}
```

Simulation output: